

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Karlo Rosenthal

**EKSPERTNI SUSTAV ZA IZBOR
RAČUNALNE OPREME PREMA
POTREBAMA KORISNIKA**

PROJEKT

TEORIJA BAZA PODATAKA

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Karlo Rosenthal

Matični broj: 0016155693

Studij: Baze podataka i baze znanja

**EKSPERTNI SUSTAV ZA IZBOR RAČUNALNE OPREME PREMA
POTREBAMA KORISNIKA**

PROJEKT

Mentor:

doc. dr. sc. Bogdan Okreša Đurić

Varaždin, kolovoz 2026.

Izjava o izvornosti

Izjavljujem da je ovaj projekt izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Ovaj projekt bavi se izradom ekspertnog sustava koji korisniku pomaže sastaviti stolno računalo od pojedinačnih komponenti prema njegovim potrebama. Sustav je implementiran u SWI-Prologu kao deduktivna baza podataka: katalog komponenti i minimalni zahtjevi pojedinih namjena zapisani su kao pohranjene činjenice, a stručno znanje o sastavljanju konfiguracije izraženo je pravilima iz kojih se novi podaci izvode zaključivanjem. Korisnik u grafičkom sučelju odabire namjenu računala, budžet te po želji neke od komponenti (procesor, grafičku karticu, memoriju i disk), dok sustav automatskim zaključivanjem predlaže najjeftinije komponente koje ispunjavaju korisnikov zahtjev. Sustav izračunava ukupnu cijenu konfiguracije, te ju uspoređuje s budžetom, upozorava na odabire slabije od preporuke te na kraju objašnjava kako je došao do prijedloga. Grafičko sučelje izrađeno je u biblioteci XPCE, a baza znanja može se ispitivati i izravnim upitima iz Prologove konzole. U radu je opisana aplikacijska domena, dan je teorijski uvod u deduktivne baze podataka, prikazan je model baze znanja te su opisani implementacija i primjeri korištenja. Na kraju je dana procjena SWI-Prologa kao platforme za ovakvu domenu i navedena su ograničenja izvedenog rješenja.

Ključne riječi: ekspertni sustav, deduktivne baze podataka, SWI-Prolog, XPCE, baza znanja, automatsko zaključivanje, konfiguracija računala

Sadržaj

1. Opis aplikacijske domene	1
2. Teorijski uvod	3
3. Model baze podataka	4
4. Implementacija	5
4.1. Baza znanja	5
4.2. Pravila zaključivanja	5
4.3. Ispis konfiguracije s objašnjenjem	6
4.4. Grafičko sučelje	7
5. Primjeri korištenja	8
5.1. Konfiguracija u cijelosti prepuštena sustavu	8
5.2. Kombinacija korisnikova odabira i upozorenja	8
5.3. Izravni upiti nad bazom znanja	9
5.4. Grafičko sučelje	9
6. Zaključak	11
Popis literature	12
Popis slika	13
Popis tablica	14
Popis isječaka koda	15

1. Opis aplikacijske domene

Aplikacijska domena ovog projekta je izbor računalne opreme prema potrebama korisnika, konkretno sastavljanje stolnog računala od pojedinačnih komponenti. Kupnja računala u dijelovima tipičan je problem u kojem prosječni kupac treba stručnjaka. Prodavač na temelju nekoliko jednostavnih pitanja (čemu će računalo služiti, koliko novca je na raspolaganju) zna predložiti smislenu kombinaciju komponenti. Upravo takvo stručno znanje želi se zapisati u bazu znanja i prepustiti računalu da samo izvodi zaključke. Osobna motivacija za izradom ovog projekta potječe upravo od te zamisli. Ideja nije nova, ekspertni sustavi za konfiguriranje računala izrađuju se još od osamdesetih godina prošlog stoljeća, kada su u industriji služili za sastavljanje računalnih sustava prema narudžbama kupaca, a i danas nastaju kao manji edukativni projekti, poput sustava za preporuku prijenosnika izrađenog u Prologu [1]. To pokazuje da je domena prikladna i za manje implementacije poput ove.

Osnovni koncepti domene su sljedeći. **Komponenta** je dio računala koji se može kupiti zasebno; svaka komponenta pripada jednoj **vrsti** (procesor, grafička kartica, memorija ili disk), ima svoj naziv, **razinu** i cijenu u eurima. Specifičnost domene je da razina nije istog tipa za sve vrste: za procesor i grafičku karticu ona je opisna, na zajedničkoj ljestvici slaba–srednja–jaka, dok je za memoriju i disk to jednostavno kapacitet u GB. **Namjena** je razlog zbog kojeg korisnik kupuje računalo (uredski rad, škola, programiranje, gaming, obrada videa), a svakoj namjeni pridružen je **zahtjev** za minimalnom razinom svake vrste komponente koju konfiguracija mora imati da bi za tu namjenu bila prikladna. **Konfiguracija** je skup od po jedne komponente svake vrste, uvećan za fiksnu osnovu (kućište, matična ploča i napajanje). Relacije među konceptima proizlaze iz navedenog: komponenta i zahtjev određuju se preko zajedničke ljestvice razina, namjena određuje zahtjev, a konfiguracija se sastoji od komponenti.

Druga specifičnost domene, koja ju čini zanimljivijom od običnog pretraživanja kataloga, jest da korisnik može, ali i ne mora, sam odabrati neke komponente. Tko se razumije u grafičke kartice, odabrat će točno određeni model, a ostalo prepustiti sustavu. Tko se ne razumije ni u što, sve prepušta sustavu. Sustav pritom mora poštovati korisnikove odluke, dopuniti ono što nedostaje, izračunati ukupnu cijenu, usporediti je s budžetom i upozoriti ako je neki korisnikov odabir ispod preporuke za odabranu namjenu. Od ekspertnog sustava se uz to očekuje i da zna objasniti svoj prijedlog [2].

Za implementaciju je odabran SWI-Prolog [3], iako su u okviru teme bili ponuđeni i drugi sustavi za automatsko zaključivanje: XSB, DES, Flora-2/Ergo Lite i PyDatalog. Odluka je donesena iz nekoliko razloga. Prvo, domena se prirodno izražava činjenicama i pravilima, a Prologov mehanizam zaključivanja izravno podržava upravo takav zapis znanja. Drugo, SWI-Prolog je od ponuđenih sustava najlakši za instalaciju na operacijskom sustavu Windows, aktivno se održava i ima daleko najbolju dokumentaciju, što je za projekt u kojem se tehnologija tek upoznaje vrlo važno. Treće, jedini od ponuđenih sustava dolazi s ugrađenom grafičkom bibliotekom XPCE, pa je bilo moguće izraditi pravu malu aplikaciju, a ne samo program za konzolu. XSB je teorijski najjači deduktivni sustav zahvaljujući tabliranom izvođenju, ali je njegova prednost vidljiva tek kod rekurzivnih upita kakvih u ovoj domeni gotovo da i nema. DES je zamišljen pr-

venstveno kao nastavni alat za čisti Datalog i u njemu se ne može izraditi interaktivna aplikacija, dok se PyDatalog praktički više ne održava. Zbog svega navedenog SWI-Prolog se pokazao kao najbolji omjer teorijske podloge i praktične upotrebljivosti za ovu domenu.

2. Teorijski uvod

Projekt se oslanja na pristup deduktivnih baza podataka. Deduktivna baza podataka razlikuje se od klasične relacijske po tome što se uz pohranjene podatke u njoj nalaze i pravila iz kojih se novi podaci mogu izvesti zaključivanjem. U takvoj se bazi stoga razlikuju dva dijela: dio s pohranjenim činjenicama, koji odgovara napunjenim tablicama relacijske baze i dio s pravilima za izvođenje, u kojem su relacije opisane, ali ne i pohranjene. Najpoznatiji jezik ovog pristupa je Datalog, podskup Prologa prilagođen radu s bazama podataka, čije su teorijske temelje detaljno opisali Ceri, Gottlob i Tanca [4]. Ključna ideja je da se relacija ne mora nužno pohraniti. Dovoljno je zapisati pravilo po kojem se njezine n-torke mogu izračunati iz drugih relacija. Time se izbjegava redundancija, a znanje o domeni postaje dio same baze.

U ovom projektu ta se podjela izravno prepoznaje. Katalog komponenti, zahtjevi namjena i ljestvica razina čine pohranjeni, činjenični dio baze znanja. To su obične činjenice, po strukturi jednake retcima relacijskih tablica. Sve ostalo su izvedene relacije: prikladnost komponente za namjenu, najjeftinija prikladna komponenta, sastav konfiguracije, njezina ukupna cijena. Nijedna od tih relacija nije pohranjena. Svaka se u trenutku upita izvodi iz činjenica primenom pravila. Prolog pritom koristi zaključivanje unatrag (engl. *backward chaining*), odnosno od postavljenog cilja traži pravila i činjenice kojima ga može dokazati. To je i uobičajeni način rada ekspertnih sustava izgrađenih u Prologu [2]. Za sustav ovakvog tipa to je prirodan smjer, jer se odgovor traži na konkretno pitanje korisnika, za razliku od sustava koji ulančavanjem unaprijed iz ulaznih podataka grade sve moguće zaključke.

U odnosu na druge pristupe obrađene na kolegiju, deduktivni pristup za ovu domenu ima nekoliko prednosti. Naprema relacijskim bazama, prednost je što se stručno znanje („za gaming treba jaka grafika”) ne mora ugraditi u aplikacijski kod ili složene SQL upite, nego stoji u bazi znanja uz podatke, čitljivo i lako izmjenjivo. Prema aktivnim bazama podataka, gdje se reaktivno znanje izražava okidačima, prednost je što deduktivna pravila nemaju bočnih učinaka pa ih je lakše razumjeti i kombinirati. Objektno-relacijski i objektno-orijentirani pristupi rješavaju druge probleme (složene tipove i hijerarhije), koji u ovoj maloj i pravilnoj domeni ne postoje. Glavni nedostaci deduktivnog pristupa su praktične naravi. Tehnologija je slabije raširena u praksi, nema standardiziranu potporu za trajnu pohranu velikih količina podataka niti za višekorisnički rad, a učinkovitost kod vrlo velikih kataloga ovisila bi o naprednijim tehnikama izvođenja poput tabliciranja. Za katalog od tridesetak komponenti nijedan od tih nedostataka ne dolazi do izražaja, pa prednosti jasno prevladavaju.

3. Model baze podataka

Bazu znanja čini pet relacija s pohranjenim činjenicama i skup predikata koji se iz njih izvode pravilima. Model se može opisati na način uobičajen za konceptualno modeliranje, uz napomenu da u Prologu relacije odgovaraju predikatima, a n-torke činjenicama. Središnja relacija je *komponenta/4*, čiji je ključ naziv komponente. Njezin atribut razina određuje se preko relacije *razina/2* (veza 0..1 : N, jer se opisne razine preslikavaju u brojeve, dok su kapaciteti brojevi sami po sebi). Relacija *zahtjev/5* s ključem namjena vrednuje svoje minimume preko iste ljestvice razina (veza N : N). Dinamička relacija *odabrano/2* odnosi se na točno jednu komponentu (veza N : 1), a *odgovor/2* preko odabrane namjene određuje mjerodavni zahtjev (veza N : 1). Predikati koji se izvode pravilima nadovezuju se na sve navedene relacije.

Statički dio modela čine relacije *komponenta/4* (katalog od 33 komponente), *zahtjev/5* (minimalni zahtjevi pet namjena) i *razina/2* (preslikavanje opisnih razina u brojeve radi usporedbe). Dinamički dio čine relacije *odgovor/2* i *odabrano/2*, koje nastaju radom korisnika u sučelju i brišu se pri svakom novom sastavljanju; one igraju ulogu radne memorije ekspertnog sustava [2]. Pregled svih relacija s pohranjenim činjenicama dan je u tablici 1.

Tablica 1: Relacije s pohranjenim činjenicama baze znanja (izradio autor)

Relacija	Vrsta	Opis
komponenta/4	statička	vrsta, naziv, razina i cijena svake komponente iz kataloga
zahtjev/5	statička	minimalne razine komponenti za pojedinu namjenu
razina/2	statička	brojčana vrijednost opisne razine (slaba = 1 ... jaka = 3)
vrsta_komponente/1	statička	popis vrsta komponenti i redoslijed obrade
odgovor/2	dinamička	korisnikovi odgovori o namjeni i budžetu
odabrano/2	dinamička	komponente koje je korisnik sam odabrao

Izvedeni dio modela može se čitati kao lanac izvođenja. Predikat *zadovoljava_min/2* uspoređuje razinu komponente s minimumom, pri čemu opisne razine uspoređuje preko relacije *razina/2*, a kapacitete izravno. Na njemu se grade *prikladna/3* (komponenta dovoljno dobra za namjenu) i *najjeftinija/3* (najjeftinija takva komponenta). Središnji predikat *dio_konfiguracije/2* za svaku vrstu daje komponentu konačne konfiguracije: korisnikov odabir ako postoji, inače prijedlog sustava. Iz njega se izvode *ukupna_cijena/1* te pomoćni predikati za savjete i upozorenja.

4. Implementacija

Cijeli sustav smješten je u dvije datoteke: `ekspertni_sustav.pl` sadrži bazu znanja, pravila zaključivanja i ispis rezultata, a `sučelje.pl` grafičko sučelje koje učitava prvu datoteku i koristi ista pravila. U nastavku su prikazani osnovni dijelovi implementacije. Kod je namjerno pisan jednostavno, a ispisi bez hrvatskih dijakritičkih znakova radi ispravnog prikaza neovisno o postavkama sustava.

4.1. Baza znanja

Katalog komponenti zapisan je činjenicama predikata `komponenta/4`, a zahtjevi namjena činjenicama predikata `zahtjev/5`, kako prikazuje isječak koda 1.

```
1 % komponenta(Vrsta, Naziv, Razina, Cijena).
2 komponenta(procesor, 'AMD Ryzen 5 5600', srednja, 165).
3 komponenta(procesor, 'Intel Core i7-14700', jaka, 420).
4 komponenta(graficka, 'NVIDIA RTX 4060', srednja, 330).
5 komponenta(graficka, 'AMD RX 7800 XT', jaka, 520).
6 komponenta(memorija, '32 GB RAM', 32, 85).
7 komponenta(disk, '1 TB SSD', 1000, 75).

8 % zahtjev(Namjena, MinProcesor, MinGrafika, MinRam, MinDisk).
9 zahtjev(ured, slaba, slaba, 8, 256).
10 zahtjev(gaming, srednja, jaka, 16, 1000).
11 zahtjev(obrađa_idea, jaka, jaka, 32, 2000).
```

Isječak koda 1: Dio činjenične baze znanja: katalog komponenti i zahtjevi namjena

Ovakav zapis istovjetan je punjenju tablica u relacijskoj bazi, s razlikom da se atribut razina puni vrijednostima dvaju tipova: opisnim atomima za procesor i grafičku karticu te brojevima za memoriju i disk. Obje opisne vrste dijele istu ljestvicu (slaba–srednja–jaka), pa je za njihovo vrednovanje dovoljna jedna relacija `razina/2`. Ipak, dvojnost tipova zahtijevala je poseban predikat za usporedbu, opisan u nastavku.

4.2. Pravila zaključivanja

Jezgru sustava čine pravila za izvođenje prikazana u isječku koda 2. Predikat `zadovoljava_min/2` prvo provjerava je li razina opisna (postoji li u relaciji `razina/2`) i tada je uspoređuje preko brojčane vrijednosti, a u protivnom uspoređuje brojeve izravno.

Predikat `najjeftinija/3` skuplja sve prikladne komponente u listu parova cijena–naziv te sortiranjem uzima prvi, dakle najjeftiniji par. Najvažniji predikat `dio_konfiguracije/2` ima dvije klauzule koje se međusobno isključuju. Ako za vrstu postoji korisnikov odabir, on ulazi u konfiguraciju. Ako ne postoji korisnikov odabir, sustav iz namjene čita minimalni zahtjev i predlaže najjeftiniju prikladnu komponentu. Upravo se u ovom predikatu vidi bit deduktivnog pristupa. Konfiguracija nigdje nije pohranjena, nego se u cijelosti izvodi iz činjenica i pravila.

```

1  % razina komponente dostize minimum
2  zadovoljava_min(Razina, Min) :-
3      razina(Razina, _), !,
4      dovoljno(Razina, Min).
5  zadovoljava_min(Broj, Min) :-
6      number(Broj),
7      Broj >= Min.

8  % najjeftinija komponenta vrste koja zadovoljava minimum
9  najjeftinija(Vrsta, Min, Naziv) :-
10     findall(Cijena-N,
11             ( komponenta(Vrsta, N, Razina, Cijena),
12               zadovoljava_min(Razina, Min) ),
13             Lista),
14     sort(Lista, [_-Naziv|_]).

15 % komponenta koja ulazi u konacnu konfiguraciju
16 dio_konfiguracije(Vrsta, Naziv) :-
17     odabrano(Vrsta, Naziv).
18 dio_konfiguracije(Vrsta, Naziv) :-
19     \+ odabrano(Vrsta, _),
20     odgovor(namjena, Namjena),
21     minimalno(Vrsta, Namjena, Min),
22     najjeftinija(Vrsta, Min, Naziv).

```

Isječak koda 2: Središnja pravila za izvođenje u bazi znanja

4.3. Ispis konfiguracije s objašnjenjem

Rezultat zaključivanja korisniku predstavlja predikat `ispisi_konfiguraciju/0`, koji za svaku vrstu komponente ispiše odabrani dio, zbroji ukupnu cijenu, usporedi je s budžetom te doda upozorenja i objašnjenje. Isječak koda 3 prikazuje dva karakteristična dijela tog ispisa.

```

1  ispisi_dio(Vrsta) :-
2      dio_konfiguracije(Vrsta, Naziv),
3      komponenta(Vrsta, Naziv, Razina, Cijena),
4      ( odabrano(Vrsta, Naziv)
5      -> Izvor = 'korisnik'
6      ;   Izvor = 'sustav'
7      ),
8      format(' * ~w: ~w (~w) - ~w EUR  [~w]~n',
9              [Vrsta, Naziv, Razina, Cijena, Izvor]).

10 ispisi_upozorenja :-
11     forall(ispod_preporuke(Vrsta, Naziv, Min),
12            format('UPOZORENJE: ~w "~w" je ispod preporuke (barem ~w)!~n',
13                  [Vrsta, Naziv, Min])).

```

Isječak koda 3: Ispis pojedinog dijela konfiguracije i upozorenja

Uz svaku komponentu ispisuje se oznaka `[korisnik]` ili `[sustav]`, ovisno o tome tko ju je odabrao, čime je u svakom trenutku jasno što je odluka korisnika, a što prijedlog sustava. Predikat `ispisi_upozorenja/0` pomoću `forall/2` prolazi kroz sve korisnikove odabire koji su ispod zahtjeva namjene, a u slučaju prekoračenja budžeta sustav izvodi i najjeftiniju moguću konfiguraciju za odabranu namjenu te na temelju nje daje savjet.

4.4. Grafičko sučelje

Grafičko sučelje izrađeno je u biblioteci XPCE koja dolazi ugrađena u SWI-Prolog. Način rada s objektima sučelja (stvaranje naredbom `new/2`, slanje poruka naredbom `send/2` i čitanje svojstava pomoću `get/3`) preuzet je iz službenog priručnika za XPCE [5]. Prozor se sastoji od padajućih izbornika za namjenu i četiri vrste komponenti, polja za budžet, polja za ispis rezultata zaključanog za uređivanje, te gumbima za sastavljanje konfiguracije, poništavanje odabira i za izlaz. Izbornici komponenti grade se naredbom `forall/2` izravno iz baze znanja, pa proširenje kataloga ne zahtijeva izmjene sučelja. Isječak koda 4 prikazuje predikat koji se poziva pritiskom na glavni gumb.

```
1 gui_konfiguracija(Namjena, Budzet, Proc, Graf, Mem, Disk, Prikaz) :-
2     retractall(odgovor(_, _)),
3     retractall(odabrano(_, _)),
4     assertz(odgovor(namjena, Namjena)),
5     assertz(odgovor(budzet, Budzet)),
6     zapamti_odabir(procesor, Proc),
7     zapamti_odabir(graficka, Graf),
8     zapamti_odabir(memorija, Mem),
9     zapamti_odabir(disk, Disk),
10    with_output_to(string(Tekst), ispisi_konfiguraciju),
11    send(Prikaz, clear),
12    send(Prikaz, append, Tekst),
13    send(Prikaz, caret, 0),
14    send(Prikaz, scroll_to, 0).
```

Isječak koda 4: Predikat gumba za sastavljanje konfiguracije u grafičkom sučelju

Vrijedi primijetiti da grafičko sučelje ne sadrži nikakvu vlastitu logiku: ono samo prepisuje stanje izbornika u dinamičke predikate i poziva isto pravilo ispisa, čiji se izlaz naredbom `with_output_to/2` formira u tekst i prikaže u prozoru pomaknutom na početak. Time je znanje sustava ostalo na jednom mjestu, a sučelje mu je samo pristupilo. Upravo zato se ista pravila mogu ispitivati i izravnim upitima iz konzole.

5. Primjeri korištenja

5.1. Konfiguracija u cijelosti prepuštena sustavu

U prvom primjeru korisnik je u sučelju odabrao namjenu *gaming* i budžet od 1500 EUR, a sve izbornike komponenti ostavio na opciji *sustav predlaže*. Ispis sustava prikazan je u isječku koda 5.

```
1 PREDLOŽENA KONFIGURACIJA (stolno racunalo, namjena: gaming):
2   * procesor: AMD Ryzen 5 5600 (srednja) - 165 EUR [sustav]
3   * graficka: AMD RX 7800 XT (jaka) - 520 EUR [sustav]
4   * memorija: 16 GB RAM (16) - 45 EUR [sustav]
5   * disk: 1 TB SSD (1000) - 75 EUR [sustav]
6   * kuciste, matična ploča i napajanje - 150 EUR
7 UKUPNA CIJENA: 955 EUR (budžet: 1500 EUR)
8 Konfiguracija je unutar budžeta.

9 Objašnjenje:
10  - namjena "gaming" traži barem:
11    procesor srednja, grafika jaka, 16 GB RAM, 1000 GB disk
12  - komponente koje niste odabrali sustav bira kao najjeftinije
13    koje još uvijek zadovoljavaju te minimalne zahtjeve
```

Isječak koda 5: Konfiguracija za gaming kada sve komponente bira sustav

Sustav je za svaku vrstu odabrao najjeftiniju komponentu koja još zadovoljava zahtjeve namjene za gaming, pa je konfiguracija ostala znatno ispod budžeta. Uz svaku komponentu ispisana je oznaka [sustav], a na kraju je dano i objašnjenje prijedloga.

5.2. Kombinacija korisnikova odabira i upozorenja

U drugom primjeru korisnik je za namjenu *obrada videa* uz budžet od 800 EUR sam odabrao najjeftiniji procesor iz kataloga (AMD Athlon 3000G). Ispis u isječku koda 6 pokazuje kako sustav reagira na oba problema koja iz toga proizlaze.

```
1 UKUPNA CIJENA: 970 EUR (budžet: 800 EUR)
2 Konfiguracija premašuje budžet za 170 EUR!
3 UPOZORENJE: ni najjeftinija konfiguracija za namjenu
4   "obrada_vida" (1245 EUR)
5   ne stane u budžet od 800 EUR, povećajte budžet.
6 UPOZORENJE: procesor "AMD Athlon 3000G" je ispod preporuke
7   (barem jaka)!
```

Isječak koda 6: Upozorenja sustava kod preslabog procesora i premalog budžeta

Sustav je korisnikov odabir poštovao i ugradio ga u konfiguraciju, ali je upozorio da je procesor ispod preporuke za obradu videa. Uz to je izračunao da konfiguracija premašuje budžet te je, izvodeći najjeftiniju moguću konfiguraciju za tu namjenu, zaključio da uz zadani budžet zadovoljavajuće rješenje uopće ne postoji, pa je korisniku savjetovao povećanje budžeta.

5.3. Izravni upiti nad bazom znanja

Kako je riječ o deduktivnoj bazi, sva se pravila mogu ispitivati i izravno iz Prologove konzole, bez dijaloga. Isječak koda 7 prikazuje tri takva upita s odgovorima.

```
1  ?- prikladna(graficka, gaming, N) .
2  N = 'AMD RX 7800 XT' ;
3  N = 'NVIDIA RTX 4070' ;
4  N = 'AMD RX 7900 XTX' ;
5  N = 'NVIDIA RTX 4080' .

6  ?- najjeftinija(procesor, jaka, P) .
7  P = 'AMD Ryzen 7 7700X' .

8  ?- najjeftinija_konfiguracija(obrađa_videa, C) .
9  C = 1245.
```

Isječak koda 7: Izravni upiti nad bazom znanja i odgovori sustava

5.4. Grafičko sučelje

Prethodni scenariji u praksi se provode kroz grafičko sučelje. Na slici 1 korisnik je za namjenu gaming uz budžet od 1500 EUR sam odabrao grafičku karticu NVIDIA RTX 4060, a ostale komponente prepustio sustavu. U polju rezultata vidi se da je korisnikov odabir označen oznakom `[korisnik]`, te da je konfiguracija unutar budžeta i da je sustav upozorio kako je odabrana kartica srednje razine. Dakle ispod je preporuke za gaming. Gumb *poništi odabire* vraća sve izbornike komponenti na opciju *sustav predlaže* i briše prikazani rezultat.

Sastavljanje stolnog racunala od pojedinih komponenti.

Namjena: Igre

Budzet eur: 1500

Procesor: Sustav predlaže

Graficka: NVIDIA RTX 4060

Memorija: Sustav predlaže

Disk: Sustav predlaže

Sastavi konfiguraciju Ponisti odabire Izlaz

PREDLOZENA KONFIGURACIJA (stolno racunalo, namjena: igre):

- * procesor: AMD Ryzen 5 5600 (srednja) - 165 EUR [sustav]
- * graficka: NVIDIA RTX 4060 (srednja) - 330 EUR [korisnik]
- * memorija: 16 GB RAM (16) - 45 EUR [sustav]
- * disk: 1 TB SSD (1000) - 75 EUR [sustav]
- * kuciste, matična ploča i napajanje - 150 EUR

UKUPNA CIJENA: 765 EUR (budzet: 1500 EUR)

Konfiguracija je unutar budzeta.

UPOZORENJE: graficka "NVIDIA RTX 4060" je ispod preporuke (barem jaka)!

Objasnjenje:

- namjena "igre" trazi barem:
 - procesor srednja, grafika jaka, 16 GB RAM, 1000 GB disk
- komponente koje niste odabrali sustav bira kao najjeftinije koje jos uvijek zadovoljavaju te minimalne zahtjeve
- podrazumijeva se stolno racunalo, pa je u cijenu ukljucena osnova (kuciste, matična ploča i napajanje)

Slika 1: Grafičko sučelje ekspertnog sustava (snimka zaslona, samostalna izrada)

6. Zaključak

Cilj projekta bio je izraditi ekspertni sustav za izbor računalne opreme prema potrebama korisnika koristeći tehnologiju deduktivnih baza podataka, a rezultat je funkcionalan konfigurator stolnog računala u SWI-Prologu s grafičkim sučeljem i mogućnošću izravnih upita nad bazom znanja. Pokazalo se da se domena gotovo idealno preslikava na deduktivni model. Katalog komponenti i zahtjevi namjena prirodno su činjenice, stručno znanje o sastavljanju prirodno su pravila, a tražena konfiguracija dobiva se automatskim zaključivanjem umjesto pisanjem procedura. Količina koda potrebna za samu logiku iznenađujuće je mala. Jezgru sustava čini desetak kratkih pravila, dok razmjerno najviše koda otpada na ispis i grafičko sučelje.

SWI-Prolog se kao platforma pokazao dobrim izborom. Instalacija i pokretanje su jednostavni, dokumentacija je opsežna, a ugrađeni XPCE omogućio je grafičko sučelje bez ijedne vanjske ovisnosti. S druge strane, tijekom izrade uočena su i ograničenja. XPCE je funkcionalan, ali vizualno zastario i slabo dokumentiran primjerima, pa je izrada sučelja tražila više eksperimentiranja od ostatka sustava. Sama implementacija također ima svoje granice, baza znanja ne modelira međusobnu kompatibilnost komponenti (podnožje procesora, standard memorije), katalog i cijene su statični, a razine komponenti su grubo diskretizirane na tri stupnja. Nijedno od tih ograničenja nije posljedica deduktivnog pristupa i sva su rješiva proširenjem baze znanja, primjerice dodavanjem relacije kompatibilnosti i pravila koja je provjeravaju.

Ukupno gledano, procjena tehnologije je pozitivna. Za domene u kojima je težište na znanju i pravilima, a količina podataka umjerena, deduktivni pristup u SWI-Prologu omogućuje čitljivu, lako proširivu i vrlo jednostavnu implementaciju kakvu bi u klasičnim tehnologijama bilo znatno teže postići.

Popis literature

- [1] prajjawal05. „Laptop Recommendation Expert System,” pogledano 17. kolovoza 2026.
adresa: <https://github.com/prajjawal05/LaptopRecommendation-ExpertSystem>
- [2] D. Merritt, *Building Expert Systems in Prolog*, adresa: <https://www.amzi.com/ExpertSystemsInProlog/index.php>, New York, 1989. pogledano 17. kolovoza 2026.
- [3] J. Wielemaker, T. Schrijvers, M. Triska i T. Lager, *SWI-Prolog*, 2012. DOI: 10.1017/S1471068411000494 pogledano 17. kolovoza 2026.
- [4] S. Ceri, G. Gottlob i L. Tanca, *What You Always Wanted to Know About Datalog (And Never Dared to Ask)*, 1989. DOI: 10.1109/69.43410 pogledano 17. kolovoza 2026.
- [5] J. Wielemaker i A. Anjewierden. „Programming in XPCE/Prolog,” pogledano 17. kolovoza 2026. adresa: <https://www.swi-prolog.org/download/xpce/doc/userguide/userguide.pdf>

Popis slika

1. Grafičko sučelje ekspertnog sustava (snimka zaslona, samostalna izrada) 10

Popis tablica

1.	Relacije s pohranjenim činjenicama baze znanja (izradio autor)	4
----	--	---

Popis isječaka koda

1.	Dio činjenične baze znanja: katalog komponenti i zahtjevi namjena	5
2.	Središnja pravila za izvođenje u bazi znanja	6
3.	Ispis pojedinog dijela konfiguracije i upozorenja	6
4.	Predikat gumba za sastavljanje konfiguracije u grafičkom sučelju	7
5.	Konfiguracija za gaming kada sve komponente bira sustav	8
6.	Upozorenja sustava kod preslabog procesora i premalog budžeta	8
7.	Izravni upiti nad bazom znanja i odgovori sustava	9